

Code of the Day – Find the Non-repeating Element in an Array

- We are given an array in which all the numbers except one are repeated once. In other words, there are $2N + 1$ numbers in the array where N numbers occur twice in the array. We need to find that number that occurs only once in the array.

```
// sequential implementation
#include <iostream>
int main() {
    int arr[] = {1,1,3,2,4,2,3};
    int size = sizeof(arr)/sizeof(int);
    int result = 0;
    for(int i =0 ; i < size; i++) {
        result = result ^ arr[i];
    }
    std::cout << result; // 4 occurs once
    return 0;
}
```

```
// parallel implementation
#include <iostream>
int main() {
    int arr[] = {1,1,3,2,4,2,3};
    int size = sizeof(arr)/sizeof(int);
    int result = 0;
    #pragma omp for simd reduction(^:result)
    for(int i =0 ; i < size; i++) {
        result = result ^ arr[i];
    }
    std::cout << result; // 4 occurs once
    return 0;
}
```

ECE759

High Performance Computing for Applications in Engineering

[Spring 2025]

OpenMP – Performance Issues

03/07/2025

Before we get started...

- Quick overview, things discussed last time
 - Scope of a variable: private, firstprivate, lastprivate
 - Execution synchronization aspects: barrier, critical, atomic
 - Parallel reduction: aggregate or combine a vector of elements to a single value through an operator
- Purpose of today's lecture:
 - OpenMP performance issues, including false sharing and NUMA effect
- Miscellaneous
 - Assignment #4 is released – due on **3/17 at 23:50 PM**
 - Final project proposal – due on **3/24 at 23:50 PM**

Work Plan, OpenMP

Parallel regions

Work sharing – Parallel tasks

Variable Scoping Issues

Synchronization

Performance issues

Getting Optimal OpenMP Performance is an Elusive Target

- Easy to write OpenMP code
 - In most cases, OpenMP code adds only a few lines of code to the existing codebase
- In terms of scalability for N cores, getting a speed-up of a factor of N is rare
 - Unless problem is embarrassingly parallel, where no dependencies exist; but this is not always possible
- Reasons for performance not being what you hope for (discuss in next slides):
 - Too much sequential code in your program
 - Too much communication
 - Load imbalance
 - Synchronizations overhead
 - Compiler optimization
 - Try to understand how your code interacts with the cache
 - Non-uniform memory access (NUMA) issues

Too much sequential code in your app

- Beware of Amdahl's law
 - The maximum speed-up you can get is up-bounded by the part that is parallelizable
- Thinking within the context of OpenMP
 - Even if in parallel region, code may not run in parallel due to the use of `master`, `barrier`, `single`, and `critical` directives
- Bottom line: If your algorithm/solution is sequential in nature, maybe it's time to change it
 - A depth first search (DFS) algorithm is less easy to parallelize than a breadth first search (BFS) one

Too much communication [that you fail to notice]

- On a shared memory machine, OpenMP can incur many intensive memory operations
 - Memory accesses are expensive (unless cached)
 - Can take ~100 cycles for a main memory access compared to 1-5 cycles for a cache memory access
- Unlike distributed computing, OpenMP communication is shared throughout the program
 - Most data transfers across memory hierarchies (cache, main memory, secondary memory storage) are handled by hardware, transparent to programmers – [hard to pin down](#)
- A good programmer should always ask “whether this line of code is expensive or not”
 - Ex: does this line of code have temporal or spatial locality?

Load Imbalance

- Load imbalance: one thread gets too much work, while the others idle waiting for it
 - Ex: The runtime of work at iteration i is proportional to i
 - As a result, one thread might become the bottleneck and hamper the overall performance
- Happens often when you use the default schedulers of `for`, `sections` & `tasks` constructs
 - Recall the default scheduler of parallel-for is static, assuming per-iteration work is even across all
 - You should study your workload and experiment different `schedule(runtime)`
 - static scheduling algorithm, dynamic scheduling algorithm, and guided scheduling algorithm
- If no good scheduling policy is available, try to fix load balancing problem in your algorithm
 - Are you asking one thread to do too much work than other threads?
 - Are you asking one iteration to do too much work than other iterators?

Synchronization is costly. Avoid if possible.

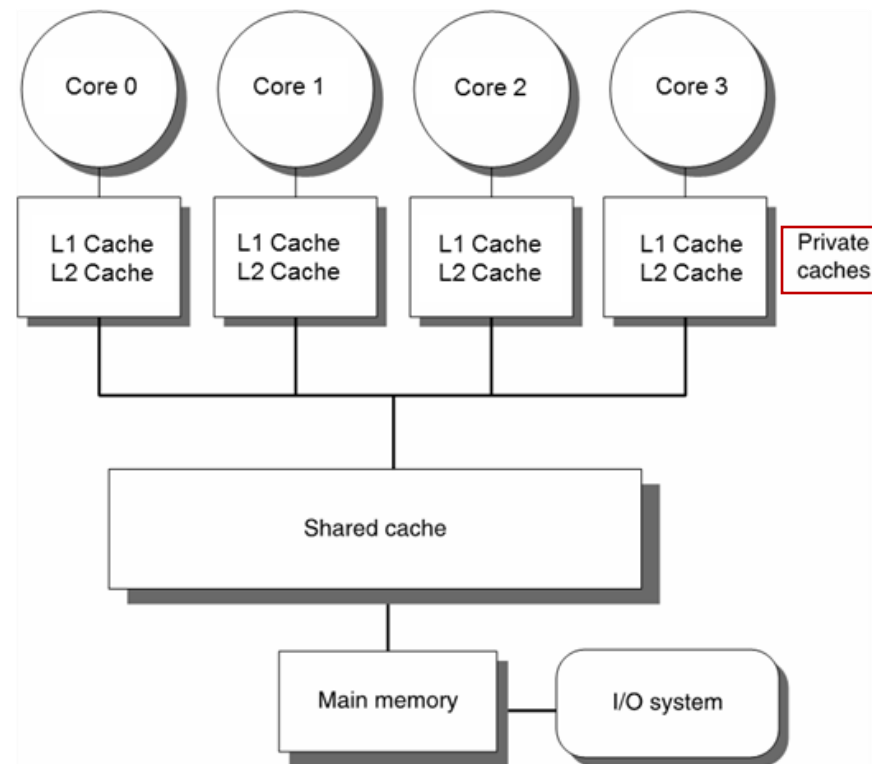
- Barriers can be very expensive as it is typically implemented using lock (`std::mutex`)
 - Failing to lock the mutex is an expensive operation since the thread will be pre-empted by the operating system to sleep waiting for the lock to be released by another thread (typically, 100–1000 cycles)
- Recall that there are many implicit barriers, such as `for`, `parallel`, and `single`
- Avoid barriers via:
 - **Careful** use of the `nowait` clause between subsequent `for`
 - Why careful? Because the second `for` loop might process data that is produced by previous `for` loop
 - Parallelize at the outermost level possible
 - May require re-ordering of loops / indices
 - Use `critical` or `atomic` only when necessary as they ruin parallelism
 - Collapse embedded loops to avoid nested parallelism
 - Use other OpenMP facilities that usually come with highly optimized implementation, e. g. `reduce`

Compiler Optimization

- Sometimes the addition of parallel directives can prevent the compiler from performing sequential optimization
 - Keep in mind, compiler is very much “sequential-thinking” and has very few ideas about your parallelism
 - When the compiler sees a “parallel” construct, it will become very conservative and cautious in optimizing code around that construct
- Sometimes, it really helps to make shared data private and local to a function
 - The more hints you give to your compiler, the higher chance your compiler can optimize your code better

Understand how your code interacts with the caches

- Thus far, we have discussed how cache works within a core
 - Basically, it's a hierarchical organization to facilitate data reuse if good data locality can be implemented
 - Each core typically has a private cache (L1, L2), but ultimately data is shared by all cores
- What if core 2 changes the memory in a block that you just cached in core 0?

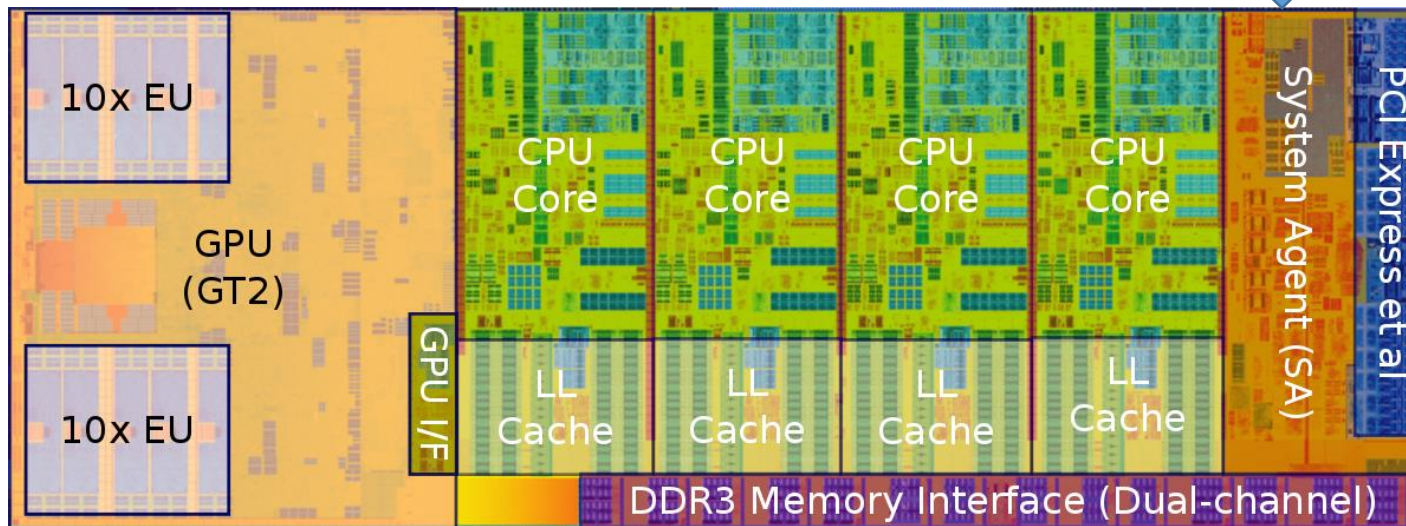


To Make Cache Work in Parallel Computing, We need Cache Coherence

Cache coherence refers to the consistency of data stored in local caches of a multi-core system, ensuring that any copy of shared data in multiple caches reflects the most recent update so that all cores can access up-to-date data.

SA: this is where the cache-coherence magic takes place
(memory controller, QPI control, snoop agent pipeline)

Haswell architecture



Multi-Core Architecture: Two Memory Aspects Coming Into Play

- We have multiple threads in a process running on multiple cores (each with its own caches)
- Two memory aspects are of important interest
 - **Memory consistency model** refers to the order in which memory reads and writes are seen by multiple processors in a shared-memory system; it defines rules for how and when changes made by one processor become visible to others, ensuring that all processors have a coherent view of the memory's state
 - **Memory coherence** refers to the consistency of data stored in local caches of a multi-core system, ensuring that any copy of shared data in multiple caches reflects the most recent update so that all cores can access up-to-date data

The Concept of “Memory Consistency”

- Code example on the right:
 - Thread_1 executes writeXY()
 - Thread_2 executes readXY()
- Question: what values can A and B assume?
- For sequential consistency (“strongly-ordered mem. model”), the only possible scenarios are
 - A=2 and B=1
 - A=20 and B=10
 - A=2 and B=10
- For weak consistency (“weakly-ordered memory model”), it is possible to have this
 - A=20 and B=1

```
int X = 1, Y = 2;
void writeXY() // run by thread 1
{
    X = 10;
    Y = 20;
}
void readXY() // run by thread 2
{
    int A = Y;
    int B = X;
}
```

Memory Consistency can be Explicitly Defined through `std::atomic`

```
std::string* ptr = 0;
int data = 0;
void producer() {
    std::string* p = new std::string("Hello");
    data = 42;
    ptr = p;
}
void consumer() {
    std::string* p2;
    while (!(p2 = ptr)) ;
    assert(data == 42); // may fail due to reordering
}
int main() {
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join(); t2.join();
}
```

```
std::atomic<std::string*> ptr = 0;
int data = 0;
void producer() {
    std::string* p = new std::string("Hello");
    data = 42;
    ptr.store(p, std::memory_order_release);
}
void consumer() {
    std::string* p2;
    while (!(p2 = ptr.load(std::memory_order_acquire))) ;
    assert(data == 42); // never fail
}
int main() { // parallelly run prod/cons
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join(); t2.join();
}
```

The Concept of “Cache Coherence”

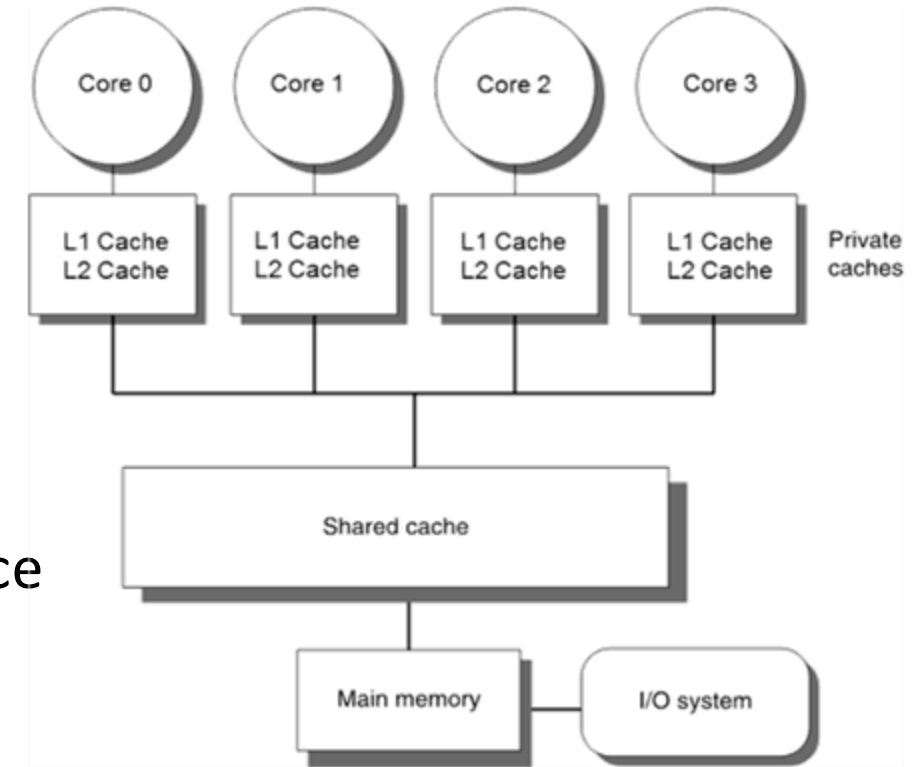
- Cache coherence is a mechanism, a hardware protocol to ensure that memory updates propagate to other cores; cores will then be able to agree on the values of information stored in memory, as if there were no cache at all
- Below, assume 32 threads work on 32 cores to perform a parallel-sum reduction

```
#pragma omp parallel for
for (int i = 0; i < 1000000; i++) {
    #pragma omp atomic
    a += input[i];
}
```

- It is possible the variable “a” will be kept in 32 copies by the 32 threads
- When one thread updates “a”, other threads **must** see the newest “a” or the reduction won’t be correct
- This is what we call “cache coherence”

Caches Coherence Mechanisms

- Cache coherence (CC) doesn't come for free
 - Though CC is typically handled by the hardware itself
 - Requires non-trivial hardware to support a CC protocol
- Significant **overhead** associated with implementing CC
 - One of the main reasons why OpenMP doesn't scale
- Two established approaches for enforcing cache coherence
 - **Directory-based**
 - **Snooping-based**

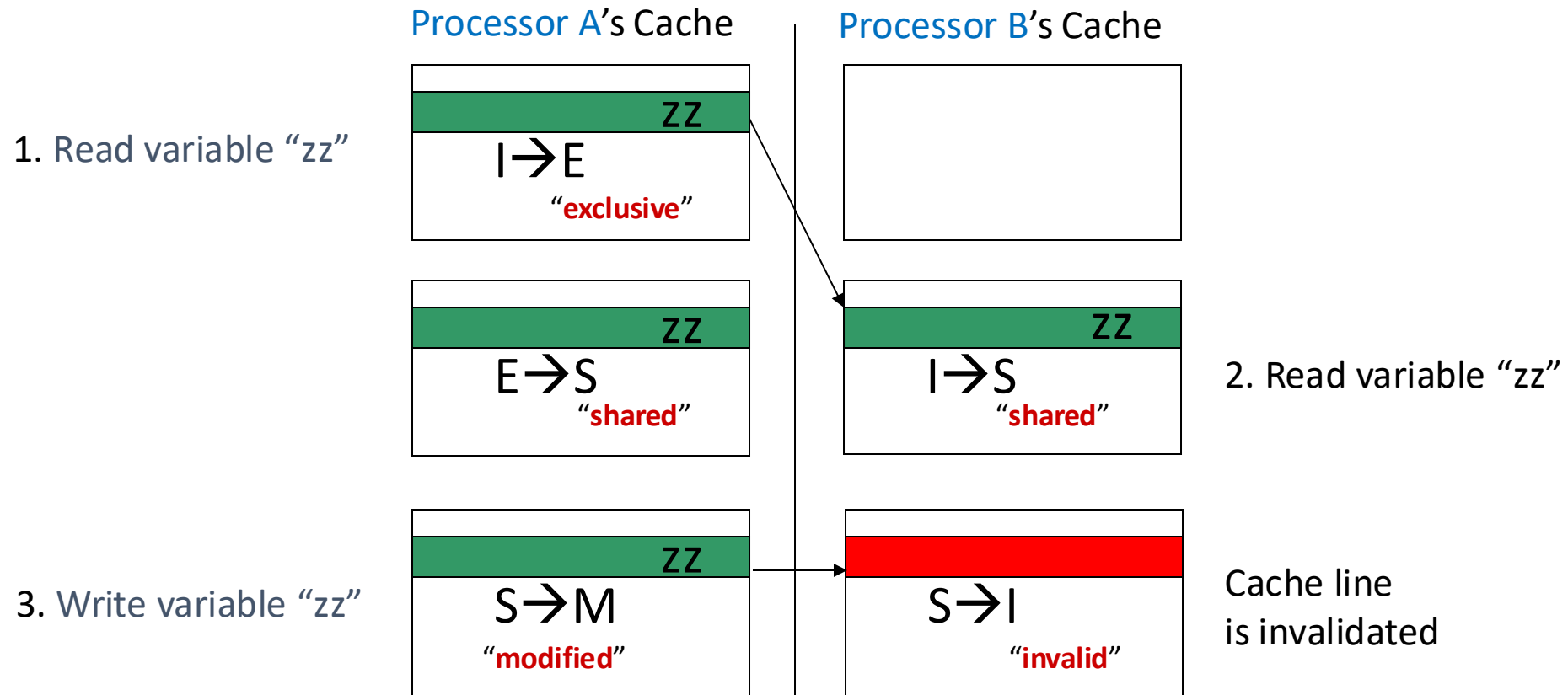


Cache Coherence Mechanisms: Directory vs Snooping Based

- Directory-based cache coherence: A central directory keeps track of the status of each memory block (whether it's cached, and if so, which processors have copies)
 - The directory maintains the state (e.g., shared, modified) and the list of caches holding the block
 - When a processor requests a block, the directory coordinates with other caches to ensure coherence through invalidating or updating copies as needed
- Snooping-based cache coherence: All processors “snoop” or monitor a common communication bus to keep track of memory accesses made by other processors
 - When a processor wants to read or write data, it checks the bus for relevant broadcasts from other processors
 - If a processor detects that another has modified a shared data block, it updates or invalidates its own cached copy to ensure coherence

MESI: Invalidation-Based Coherence Protocol

- Cache lines have **state** bits used to characterize state of one cache line
- **MESI** Protocol has four states: **M: Modified**, **E: Exclusive**, **S: Shared**, **I: Invalid**



The four states in which a cache line (CL) can find itself

[**NOTE**: clean/dirty states: Think of them in relation to what is stored in CL vs. in main memory]

- **Modified (M)**
 - CL present only in the current cache, and there's discrepancy w.r.t. what's in main memory
 - Cache required to write back the data to main memory at some time in the future, before permitting any other read of the (no longer valid) main memory block
 - NOTE: The write-back changes the CL to (S)
- **Exclusive (E)**
 - CL present only in this cache and it's clean; i.e., it matches what's in main memory
 - NOTE: Might change to (S) in response to a read request. Might change to (M) when writing to it
- **Shared (S)**
 - CL in this cache is clean; i.e., it matches what's in main memory. CL stored also in other cache[s] of the machine.
 - NOTE: Might changed to (I) due to a write in another cache of the machine
- **Invalid (I)**
 - CL is invalid; i.e., it should not be used

The “False Sharing” Pitfall

- Recall cache lines typically are 64 byte long (can store, for instance, 16 integers)
 - **IMPORTANT NOTE:** as soon as *one* byte in a cache line is changed, the entire cache line becomes dirty – Why? Because cache line is the basic unit of work in the cache architecture
- **False sharing:** happens when two threads are both writing into **different** locations within the **same** cache line
 - One write by Processor A will invalidate the cache line on Processor B
 - Memory transactions will be serialized as they all happened on the same cache line
 - Ex:

```
#pragma omp parallel for schedule(dynamic, 1)
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

False Sharing is a/the Major Source of Scalability Problem

- Parallel update that happen on the same cache line will serialize the transactions
- “Extremely” poor performance – even worse than the sequential version as you need to take into account the overhead of threading ...
- High numbers of cache misses among multiple threads

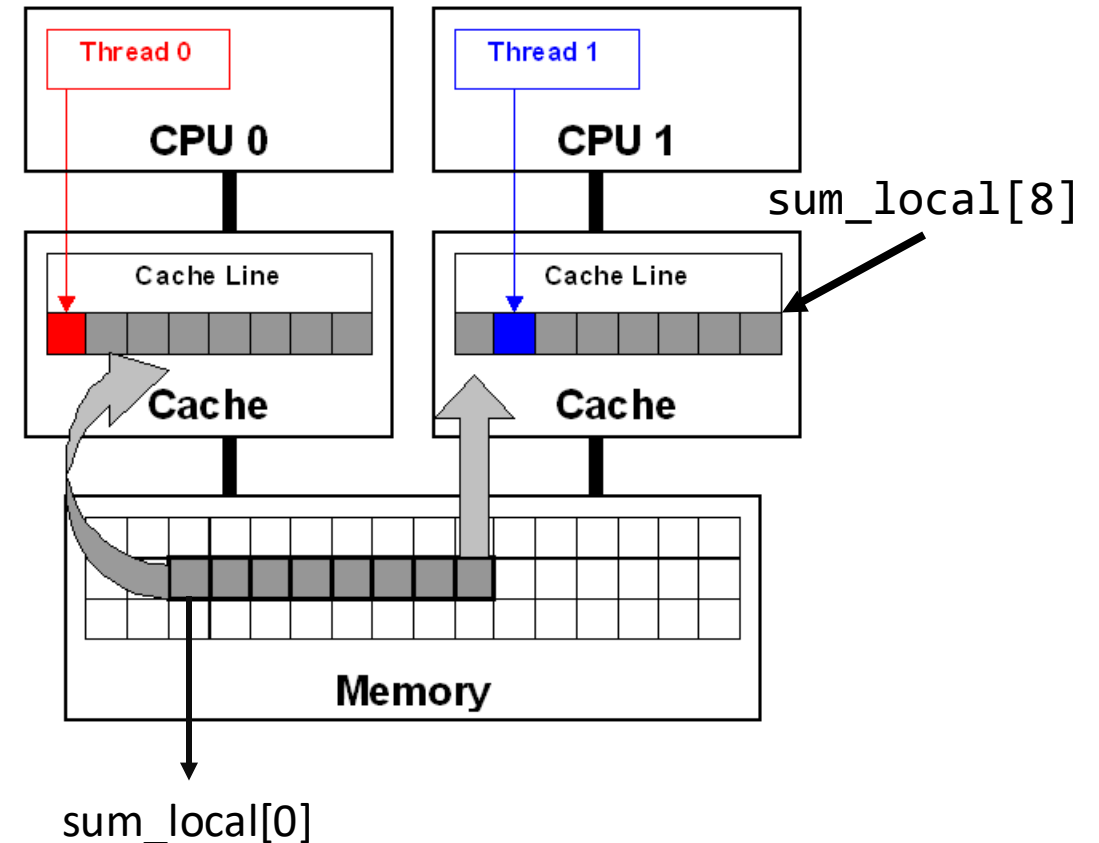
False Sharing Example [1/2]

```
double sum_local[8];
double sum = 0.0;

#pragma omp parallel num_threads(8)
{
    int me = omp_get_thread_num();
    sum_local[me] = 0.0;

    for (int i = 0; i < N; i++)
        sum_local[me] += x[i] * y[i];

    #pragma omp atomic
    sum += sum_local[me];
}
```



How can this instance of false sharing be fixed?

False Sharing Example [2/2]

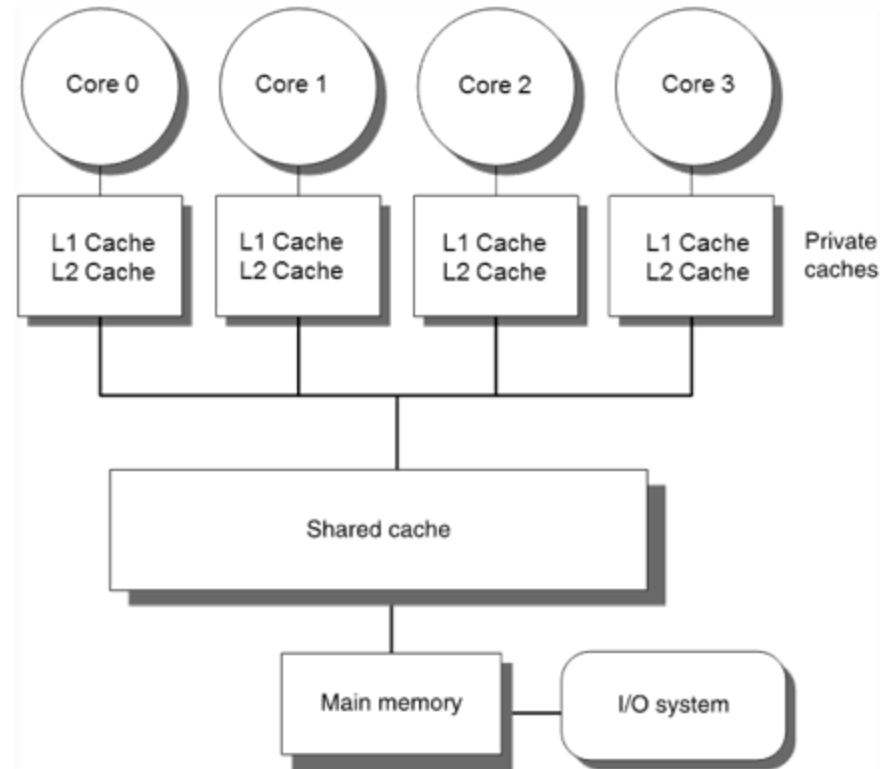
- Explicitly align your data address with the cache line size using [C++ alignas](#)
 - If data's addresses are a multiple of "cache line size", then they won't reside on the same cache line
 - Ex: data1 starts on 0, data2 starts on 64, data3 starts on 128, data4 starts on 192

```
// hardware_constructive_interference_size is typically 64 byte (cache line size)
struct alignas(hardware_constructive_interference_size) OneCacheLiner // occupies one cache line
{
    std::atomic_uint64_t x{};
    std::atomic_uint64_t y{};
} oneCacheLiner;

struct TwoCacheLiner // occupies two cache lines
{
    alignas(hardware_destructive_interference_size) std::atomic_uint64_t x{};
    alignas(hardware_destructive_interference_size) std::atomic_uint64_t y{};
} twoCacheLiner;
```


[New subtopic] Revisit the SMP model

- Up to this point, we are not concerned with the architecture of a shared memory access
 - Model embraced so far is symmetric multi-processing (SMP)
 - All threads/cores have similar overhead when accessing shared memory



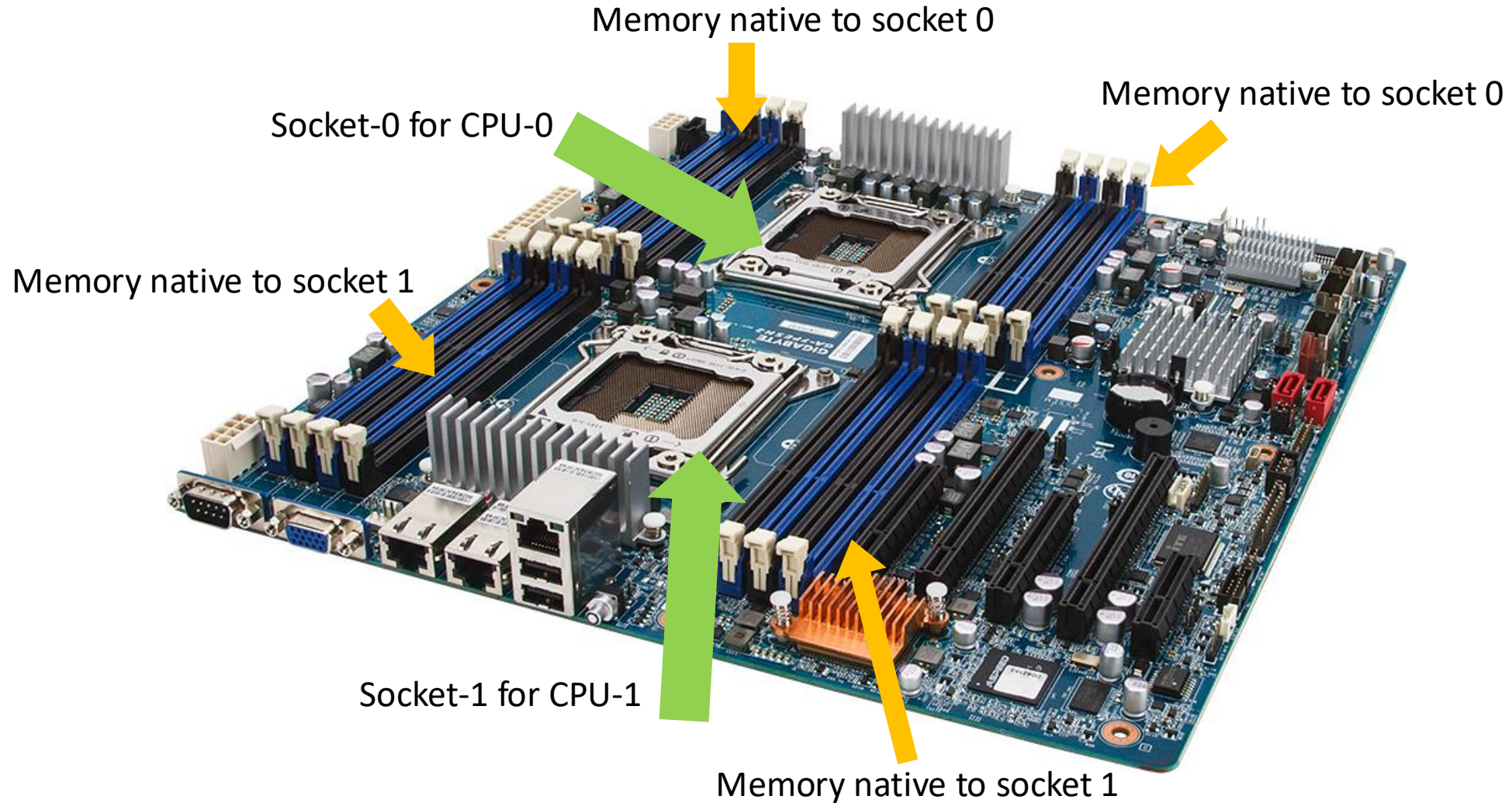
SMP is indeed a dated model, given today's high-end hardware

- Today's server/cluster power nodes have many CPUs (hundreds), each with many cores
 - **server node**: used a lot by Amazon and Facebook for database manipulations
 - **cluster node**: mostly used by Oak Ridge or Argonne National Lab in their supercomputers
- SMP doesn't quite apply, and the hardware is almost based on *multi-socket configurations*
 - Each socket connects to a group of CPU cores
 - Multiple sockets are communicated through bus and partitioned of memory native to a core
- Multi-socket architecture is not symmetrical, and thus not all memory accesses are equal
 - A certain core has access time to certain system memory smaller than or equal to other cores on the node

Multi-Socket Configurations

[1/2]

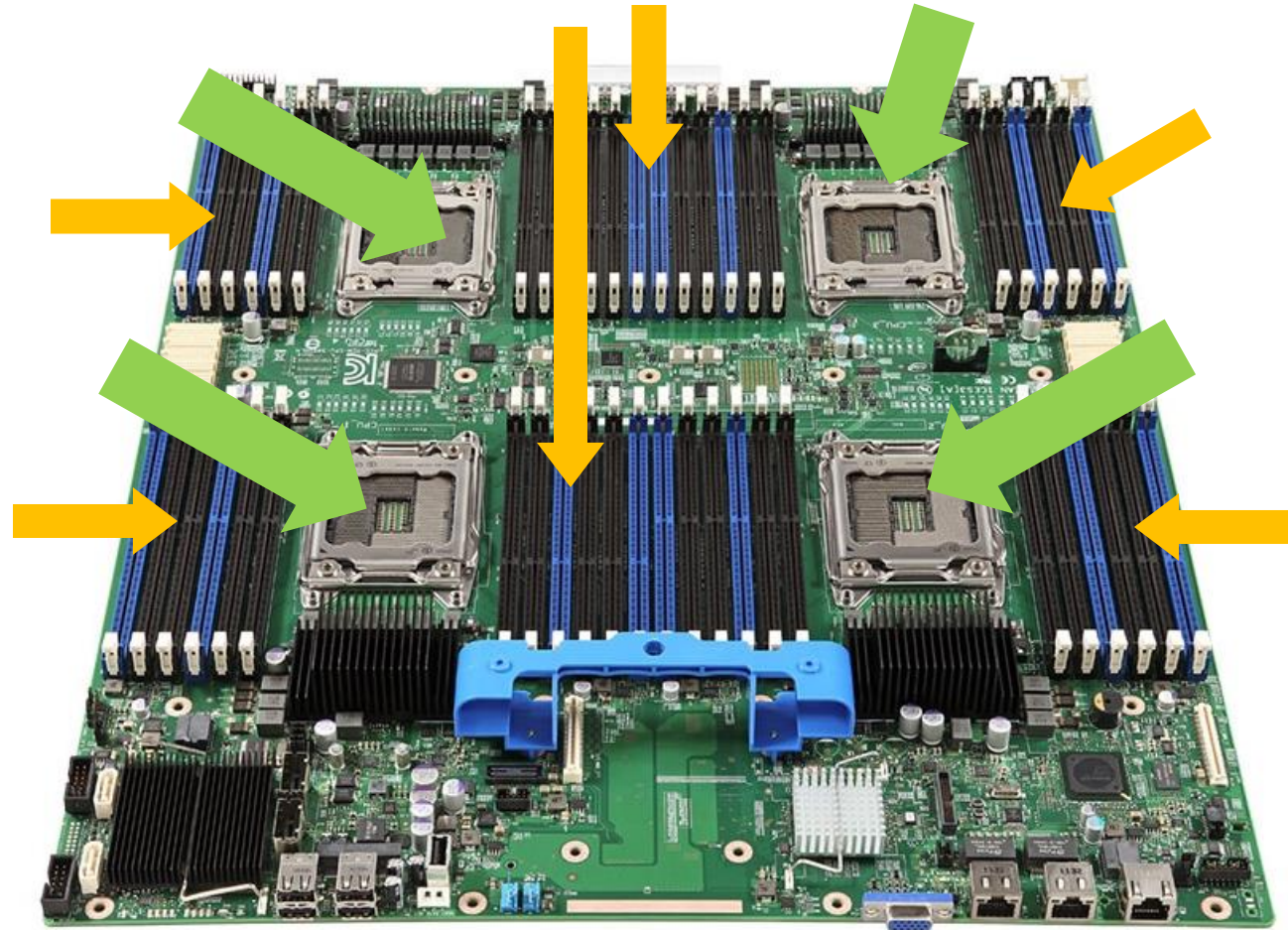
[two sockets, in this case]



Multi-Socket Configurations

[2/2]

[four sockets: Intel S4600LT2 Xeon E5-4600 Chipset-C600-A Socket-R LGA-2011 1.46Tb DDR3-1600MHz Server Motherboard]



→ : Memory Bank

→ : CPU Socket

Examples, Recent Layouts

- AMD Zen (EPYC 7601): **Two** socket system
 - 32 cores per processor $\Rightarrow$ 128 threads per socket
 - 8 channel memory access per socket
 - CPUs communicate with each other via high-speed bus
 - Each processor: $\approx$ \$4,400
- Intel Xeon Scalable (Platinum 8180): **Eight** socket system
 - 28 cores per processor $\Rightarrow$ 56 threads per socket
 - 6 channel memory access per socket
 - Each socket can only link directly to 3 neighboring sockets
 - Each processor $\approx$ \$5,000

An Important Concept: Non-uniform Memory Access (NUMA)

- NUMA: cost of memory access depends on which memory bank stores your data
- Example:
 - Assume four RAM memory banks (RAM_MB): RAM_MB1, RAM_MB2, RAM_MB3, RAM_MB4
 - Assume four CPU sockets: CPU_A, CPU_B, CPU_C, CPU_D
 - Also, assume that:
 - RAM_MB1 bank acts as “native memory” for CPU_A
 - ...
 - RAM_MB4 bank acts as “native memory” for CPU_D
- From the shared-memory programming perspective, any CPU can access any memory bank
 - However, thread running on a CPU_A core takes less time to access data stored in bank RAM_MB1 than in RAM_MB3

The `lscpu` command will Reveal Your NUMA Architecture

[interesting “affinity” information highlighted in bold red]

```
ME759node ~> lscpu
```

```
Architecture:      x86_64
CPU op-mode(s):    32-bit, 64-bit
Byte Order:        Little Endian
CPU(s):           32
On-line CPU(s) list: 0-31
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s):        2
NUMA node(s):     2
Vendor ID:         GenuineIntel
CPU family:         6
Model:              63
Model name:         Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
Stepping:           2
```

(output of `lscpu` continued
in the block to the right)



```
CPU MHz:           3116.890
CPU max MHz:        3400.0000
CPU min MHz:        1200.0000
BogoMIPS:           5187.98
Virtualization:     VT-x
L1d cache:          32K
L1i cache:          32K
L2 cache:           256K
L3 cache:           20480K
NUMA node0 CPU(s): 0-7,16-23
NUMA node1 CPU(s): 8-15,24-31
```

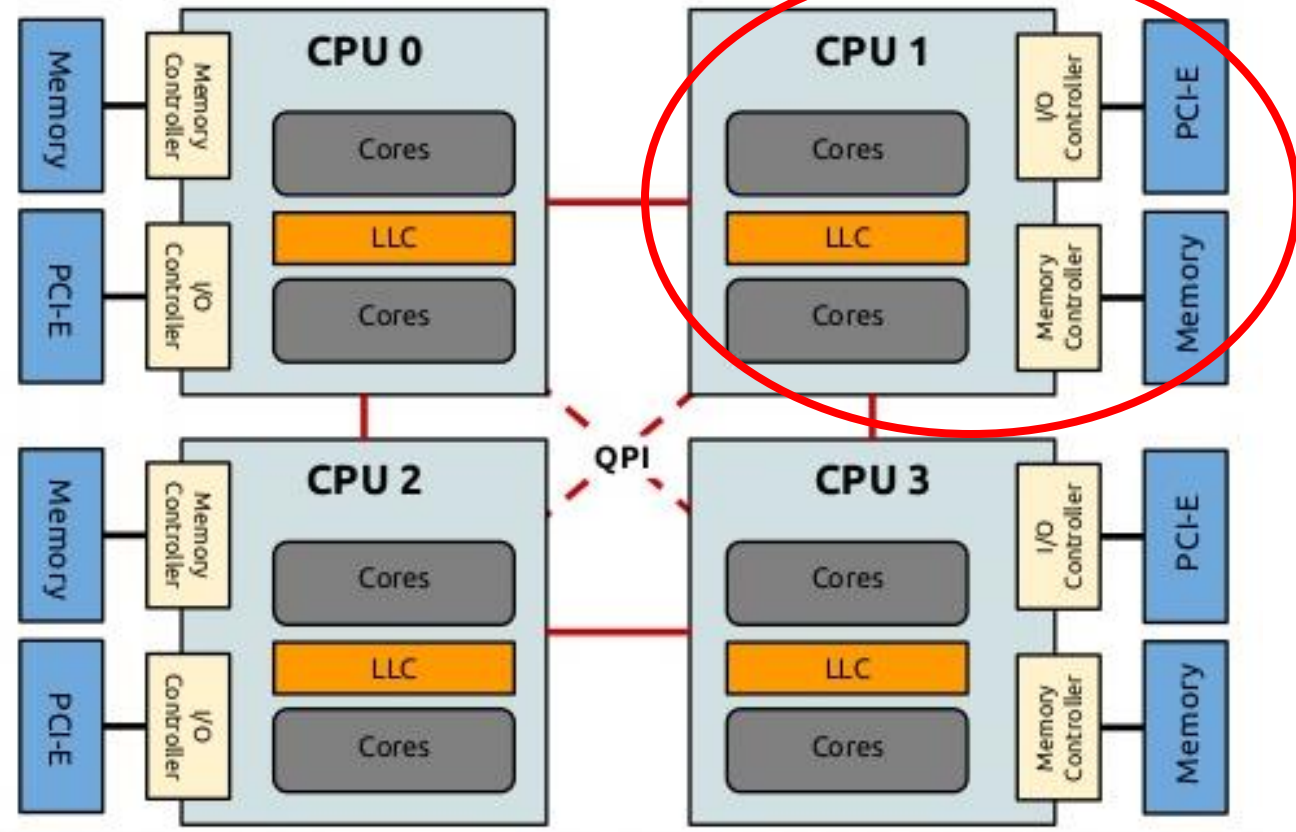
The NUMA Factor

- Defined as the ratio between the largest and shortest *average* amount of time for a thread running on a particular core to reach data in memory
 - A low NUMA factor is desirable, it suggests not much of a difference which bank data is stored
 - A high NUMA encountered in expensive hardware solutions that have hundreds to thousands of CPUs yet they have a **shared** memory layout
 - SGI's Altix back in the day
 - Less common today
 - A system w/ NUMA factor = 1 is an SMP system – memory access time is the same for each CPU

NUMA – A Schematic [Intel specific]

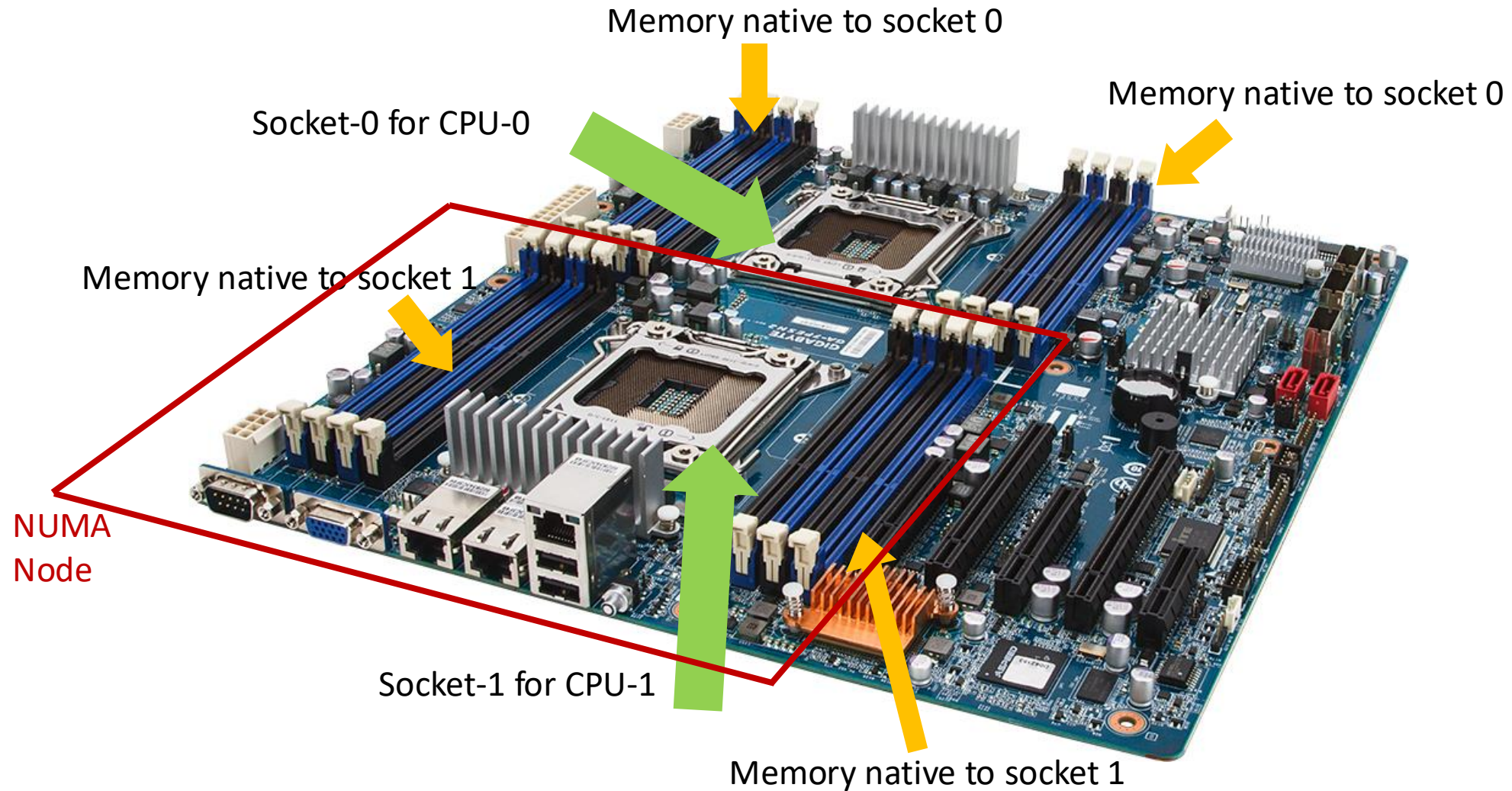
- The right image shows a motherboard w/ four sockets
- **NUMA node** is a group of processors and memory bank next to it
- QPI: QuickPath Interconnect (more later; also, replaced by Ultra Path Interconnect more recently (Xeon Skylake, 2017))

CPU architecture (Intel Sandy Bridge)



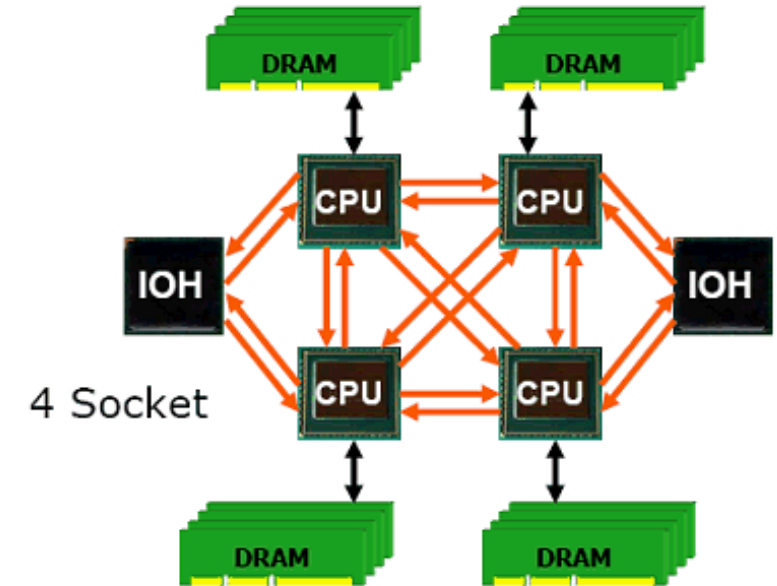
Although in the schematic memory looks distributed across cores, they are all accessible by these cores (shared memory programming model)

How a NUMA Node Looks in Your Hardware



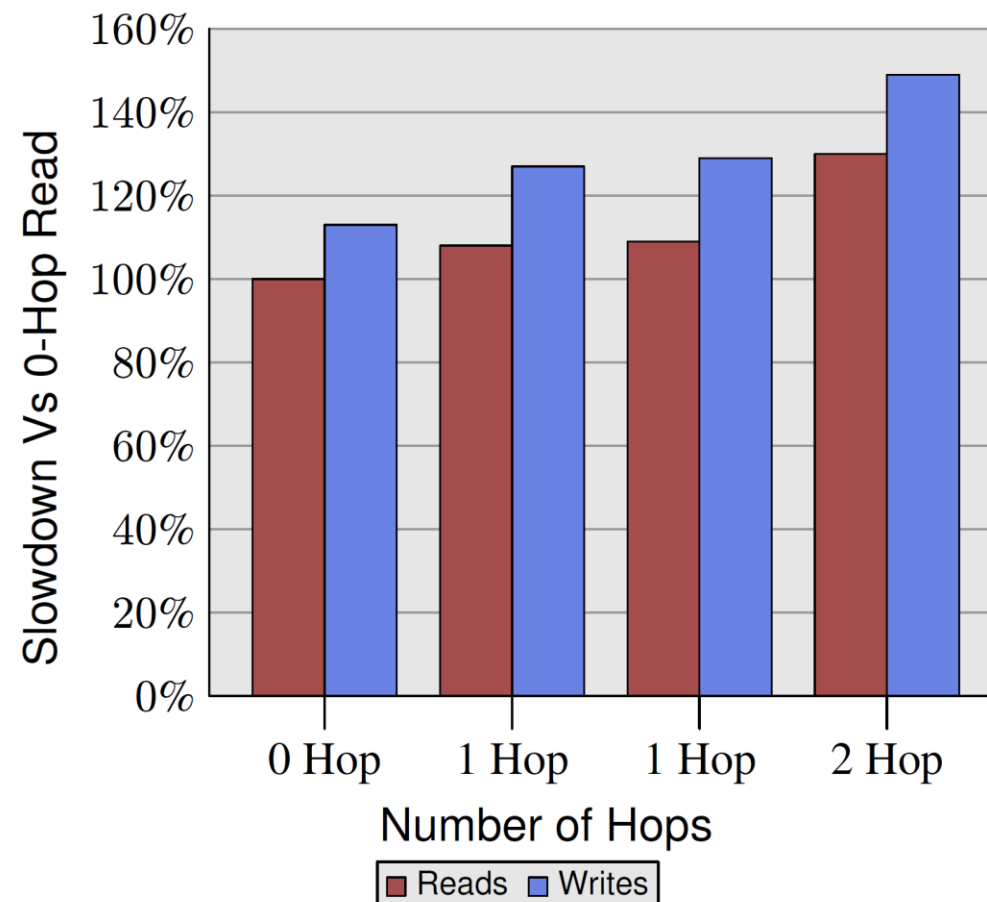
Point-to-Point Communication between NUMA Nodes

- Each CPU can access data stored in a different NUMA node
 - Happens over a very fast interconnect, which is like a local network
 - AMD's solution name: [Hyper Transport \(HT\)](#)
 - Intel's solution name: [QuickPath Interconnect \(QPI\)](#) (more recently: Ultra Path Interconnect ([UPI](#)))
- The fast interconnect (local network): shown in orange in picture
- Possible topologies associated with this “local network”:
 - 2D-torus
 - 2D-lattice (only moves in x or y direction – like 2D grid)
 - 3D-Hypercube (3D version of lattice)
 - ... still an active research area in computer network



NUMA Performance Hit, order of magnitude

- What sort of slowdown should one expect if accessing memory outside a NUMA node?
- Approximate values:
 - 20% slowdown for reads
 - 30% slowdown for writes
- Values above depend on the number of hops required to reach memory bank
 - Closer NUMA nodes lead to lower values



How does the Operating System Deal with NUMA?

1) When a thread `mallocs` memory, how should this memory be allocated?

- Examples:
 - All memory allocated in one NUMA node
 - Memory split in chunks, each NUMA node being associated with one such chunk (“striping the memory”)

2) When a thread runs an instruction, what cores should it run on?

- Example:
 - A thread w/ thread id 3 used to execute on Core #X, but then it runs on Core #Y, on a different NUMA node – the execution order is completely decided by the OS depending on the runtime/workload
 - Switching a thread from one core to another core can incur significant runtime overhead due to thread-core migration and context switches

1) The `malloc` issue, and the “first touch” story

- Memory allocated with `malloc` (and `new`) is not necessarily set aside when `malloc` gets hit
 - That only happens when data is written to it
- **x** first touch is only by Master thread
 - Memory might be allocated in its NUMA node
- **a** touched by all threads at the same time
 - Reflect on how chunks of the array are going to be spread all over the memory, very likely in different banks
- Reflect on differences between Case 1 & 2

```
int main() {
    const int N = 4000000;
    double *x = (double*)malloc(N * sizeof(double));
    double *a = (double*)malloc(N * sizeof(double));

    for (i = 0; i < N; i++)
        x[i] = simpleFunc(i);

    omp_set_num_threads(64);
    #pragma omp parallel for schedule(static,5)
    for (i = 0; i < N; i++)
        a[i] = someFunc(x[i]);

    return 0;
}
```

Case 1

```
int main() {
    const int N = 4000000;
    double *x = (double*)malloc(N * sizeof(double));
    double *a = (double*)malloc(N * sizeof(double));

    for (i = 0; i < N; i++)
        x[i] = simpleFunc(i);

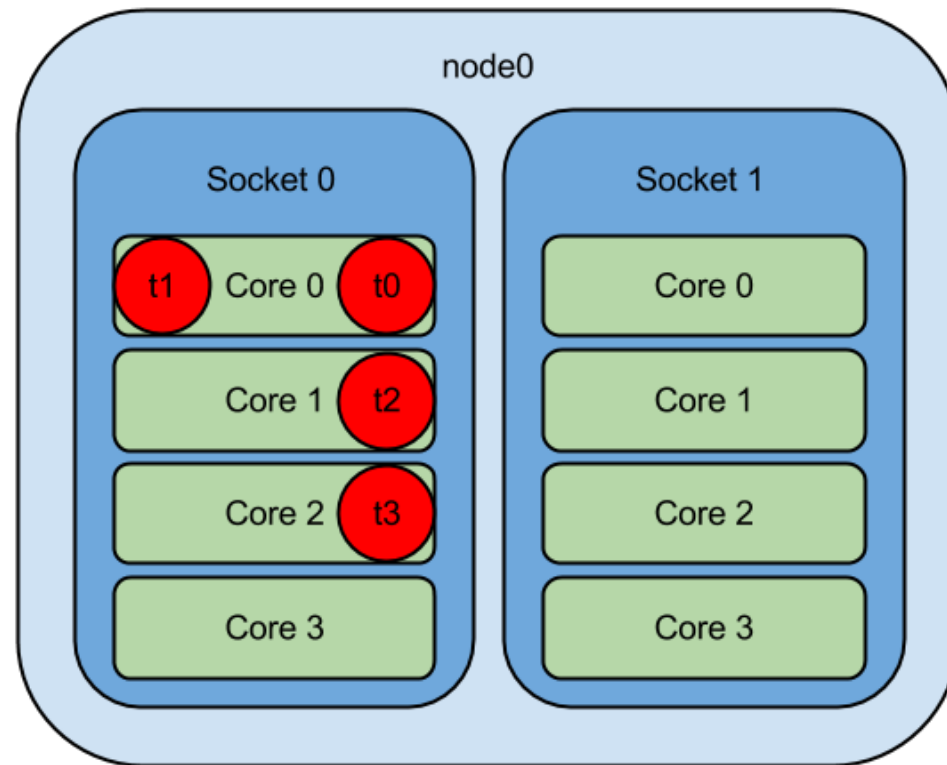
    omp_set_num_threads(64);
    #pragma omp parallel for schedule(dynamic,1)
    for (i = 0; i < N; i++)
        a[i] = someFunc(x[i]);

    return 0;
}
```

Case 2

2) Affinity

- **Affinity**: Bind a specific thread to a particular CPU core in a system
 - Improve performance by keeping tasks close to the memory or resources they frequently access
 - Minimize cache misses and reducing latency by ensuring that a process continues to run on the same core



Controlling affinity in OpenMP

- Two OpenMP environment variables allow you to control affinity:
 - `OMP_PROC_BIND` – allows you to dictate a distribution policy
 - `OMP_PLACES` – allows you to control locations
- NOTE: Can accomplish via runtime function calls too

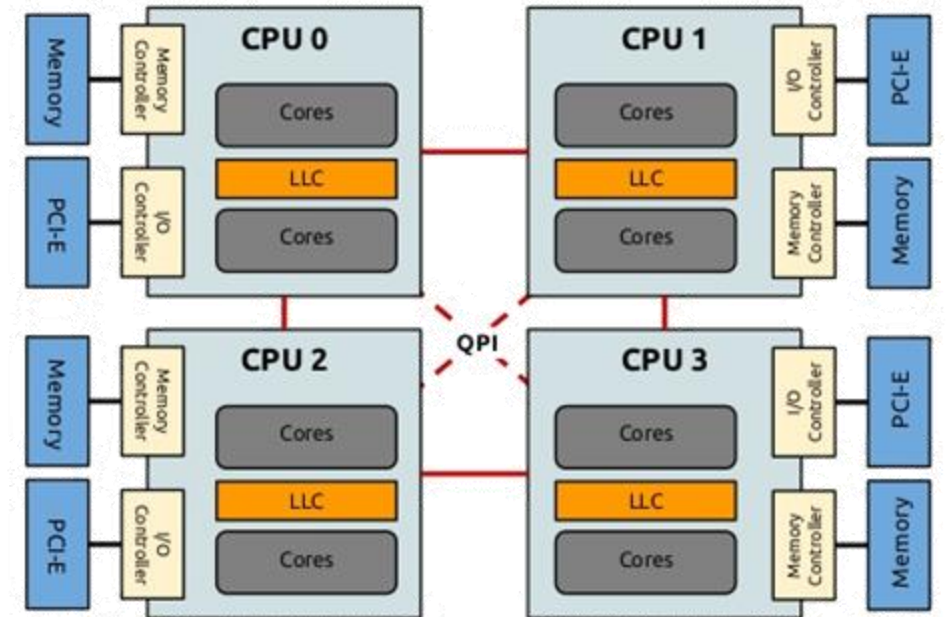
The distribution policy: **OMP_PROC_BIND**

- export OMP_PROC_BIND = master|close|spread|false|true
 - **master**: collocate threads with the master thread
 - **close**: place threads close to the master in the places list
 - **spread** (default): spread out threads as much as possible
 - **false**: set no binding
 - **true**: lock threads to a core

The **spread** Affinity Policy

- **spread** (spreads out threads as much as possible)
 - This is useful if your code is memory bound
 - Likely to improve aggregate system memory bandwidth
 - You see more cache use and less cache contention

CPU architecture (Intel Sandy Bridge)

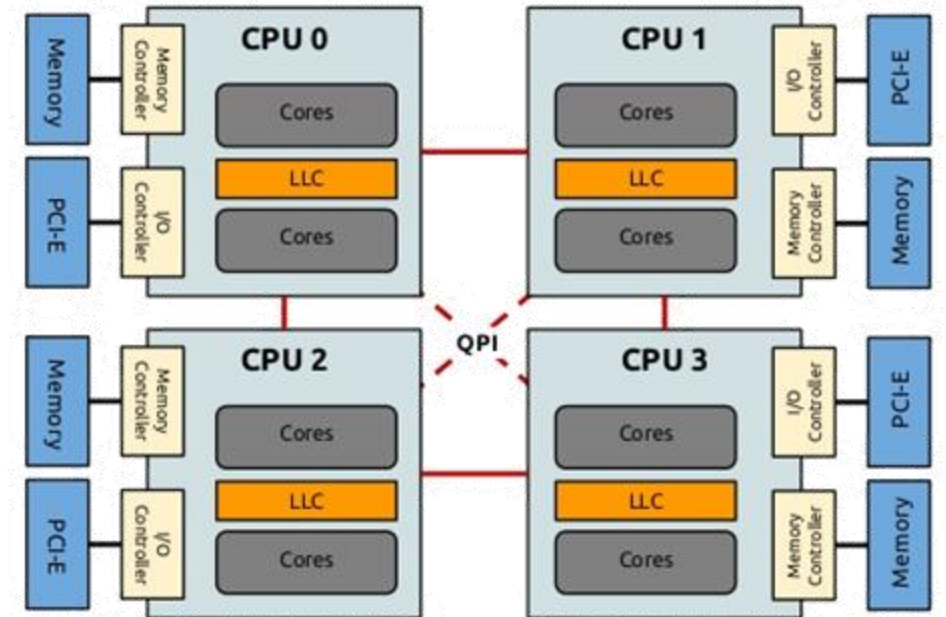


- Example: two sockets, 8 cores per socket, you ask for 8 OpenMP threads
 - Have first four OpenMP threads on one socket and use its bandwidth for talking w/ main memory
 - Have last four OpenMP threads on the other socket and use its bandwidth for talking w/ main memory

The **close** Affinity Policy

- **close** (places the threads close to the master)

CPU architecture (Intel Sandy Bridge)



- This is useful if you don't do many trips to main memory since you keep hitting the cache
 - Typically useful if you are compute bound
 - Likely to reduce synchronization costs (single, barrier, etc.)

Control Locations through OMP_PLACES

- Via environment variable: export `OMP_PLACES=<place_list>`
- Here's what `place_list` above can be:
 - A place: `{0}`
 - Enumeration: `{0},{1},{2},{3}`
 - Interval notation: `<place>:<len>:<stride>` → e.g. `{0}:4:1`
 - NOTE: `{0},{1},{2},{3}` is the same as `{0}:4:1`

List of [Linux] CPU ids
- What these numbers in red above mean:
 - with HyperThreading: list of ids of the HW-threads (virtual cores)
 - without HyperThreading: list of cores

Example [Assume 2 sockets & 4 cores per socket & **no** hyper-threading enabled]



```
export OMP_NUM_THREADS = 4
export OMP_PROC_BIND = close
```

```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```

```
export OMP_NUM_THREADS = 4
export OMP_PROC_BIND = spread
```

```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```

← bash shell

← C code



COMPACT PACKING



← thread id (tid)

← cpu id

SCATTER PACKING

Example [Assume 2 sockets & 4 cores per socket & **no** hyper-threading enabled]



```
export OMP_NUM_THREADS = 4
export OMP_PLACES = '{0},{1},{2},{3}'
```

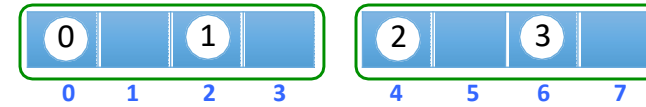
```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```

```
export OMP_NUM_THREADS = 4
export OMP_PLACES = '{0},{2},{4},{6}'
```

```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```



COMPACT PACKING



SCATTER PACKING

← thread id (tid)
← cpu id

Example [Assume 2 sockets & 4 cores per socket & hyper-threading not enabled]



```
export OMP_NUM_THREADS = 4
export OMP_PLACES = '{0}:4'
```

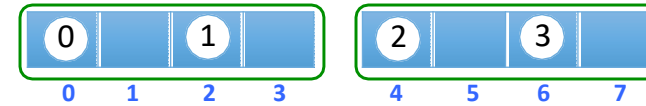
```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```



COMPACT PACKING

```
export OMP_NUM_THREADS = 4
export OMP_PLACES = '{0}:4:2'
```

```
#pragma omp parallel private(tid)
{
    tid = omp_get_thread_num();
    // code here...
}
```



SCATTER PACKING

←thread id (tid)
←cpu id

Takeaway: Affinity & cc-NUMA

- Modern manycore systems are typically NUMA-based architecture
 - Memory access time is different for each core
- Ability to control affinity is good
 - Sometimes you can avoid remote NUMA-node data access; i.e., costly local network hops
 - NOTE: overhead of cache-coherence mechanism becomes a major issue
- cc-NUMA: **cache-coherent** NUMA
 - The run time must maintain coherence between caches in different NUMA nodes